MeKo MEDTECH

Git & GitHub Workshop

Von der Idee bis in die Produktionsumgebung

Webanwendungen & Schnittstellen (EWWS)



Workshop-Agenda

Die Grundlagen hatten wir schon: Commit, Branch, Merge, Pull Request. Heute spielen wir den **kompletten Ablauf** wie bei uns üblich selbst durch — an einem Beispiel-Repo „Teamwand“.

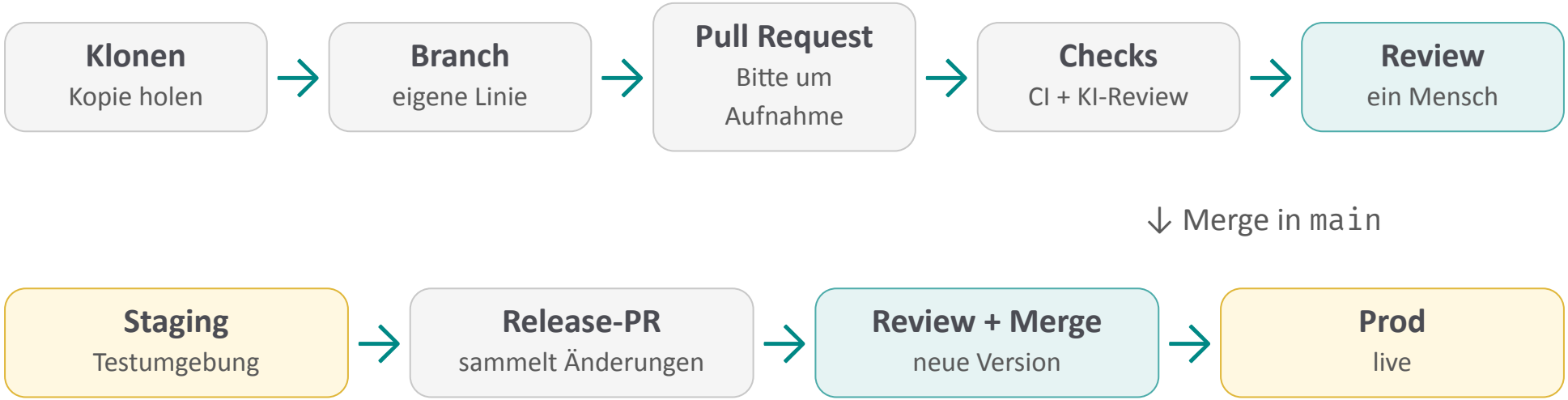
Eine Änderung pro Person: jede:r trägt sich in die Teamwand ein. Wir gehen den Weg **gemeinsam, Etappe für Etappe**.

1 · Klonen | 2 · Branch & Commit | 3 · Push & PR | 4 · Checks | 5 · Review | 6 · Merge & Staging | 7 · Release & Prod

Euer Agent tippt die Befehle — ihr sagt ihm, was er tun soll, und prüft das Ergebnis.

Der Weg einer Änderung

Jede Änderung — egal wie klein — nimmt diesen Weg:



Niemand ändert main direkt — nicht ihr, nicht der Agent, nicht die Admins.

Die Teamwand

Unser Übungsprojekt: eine Webseite mit einer Karte pro Person.

- `team.js` — die Einträge, eine Zeile pro Person
- `index.html` — die Seite, per Doppelklick öffnen
- `AGENTS.md` — die Regeln für euch **und** euren Agenten

Kein Build, kein Server — läuft direkt im Browser.

Repo

`MeKo-Christian/workshop-teamwand`

GitHub Pages — nach jedem Merge

`meko-christian.github.io/workshop-teamwand`

Release — neue Version im Footer

Release-PR mergen, dann Seite neu laden

Etappe 1: Klonen

Was liegt auf GitHub — und was holt ihr euch auf den Rechner?

1 · Klonen | 2 · Branch & Commit | 3 · Push & PR | 4 · Checks | 5 · Review | 6 · Merge & Staging | 7 · Release & Prod

Zwei Kopien: Euer Rechner und GitHub

Euer Rechner

Eure eigene, vollständige Kopie.

- Arbeitsordner mit allen Dateien
- versteckter Ordner `.git` mit der ganzen History
- hier entstehen eure Commits — auch offline

Klonen

←

nur einmal pro Rechner

danach immer wieder:

Holen (pull)

←

→

Hochladen (push)

GitHub („origin“)

Das Original, für alle erreichbar.

- alle Dateien und die ganze History
- **nur hier:** Issues, Pull Requests, Reviews, CI
- „origin“ ist nur der Name, unter dem euer Rechner diese Kopie kennt

Klonen — einmal: kopiert alles und merkt sich „origin“.

Holen — vor jeder Arbeit: neue Commits von GitHub übernehmen.

Hochladen — nach dem Commit: eure Commits zu GitHub schicken.

Zwei Kopien — unabhängig voneinander

Nichts gleicht sich von selbst ab — nur wenn ihr **holt (pull)** oder **hochladet (push)**.

Ihr ändert etwas

Liegt **nur bei euch**. GitHub sieht es erst nach dem **push**.

Jemand anders ändert etwas

Liegt **nur auf GitHub**. Ihr seht es erst nach dem **pull**.

Beide ändern denselben Branch

Euer **push** wird abgelehnt: GitHub hat auf dem Branch Commits, die ihr noch nicht habt. Erst **pull** — Git führt beides zusammen. Neues auf `main` holt ihr dagegen eigens in euren Branch (Etappe 6).

Ihr löscht euren Ordner

GitHub bleibt unverändert. Neu klonen — weg ist nur, was ihr noch nicht per **push** hochgeladen habt.

Faustregel: vor der Arbeit pull, nach dem Commit push.

🛠️ Etappe 1: Klonen

Sagt eurem Agenten (ca. 5 Minuten)

1. **„Klone MeKo-Christian/workshop-teamwand in meinen Projektordner.“**
2. **„Zeig mir die letzten fünf Commits.“**
3. **„Welche Issues sind mir auf GitHub zugewiesen?“**

Selbst prüfen: `index.html` im Browser öffnen — die Seite läuft bei euch lokal.

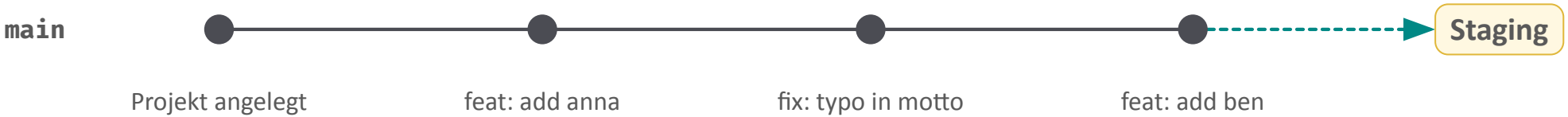
Etappe 2: Branch & Commit

Auf eigenem Branch arbeiten — vorerst nur auf eurem Rechner.

1 · Klonen | **2 · Branch & Commit** | 3 · Push & PR | 4 · Checks | 5 · Review | 6 · Merge & Staging | 7 · Release & Prod

Was ist `main`?

`main` ist die **Hauptlinie** des Projekts: der Stand, auf den sich alle geeinigt haben. Beim Klonen bekommt ihr genau diesen Stand.



Nur ein Name

Der Name ist frei wählbar. Früher hieß die Hauptlinie meist `master` — wir haben uns für `main` entschieden.

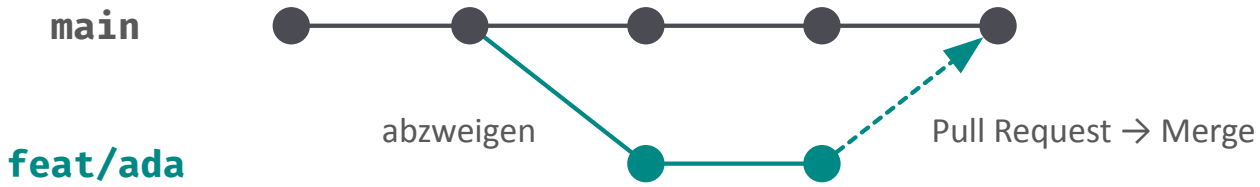
Geht auf Staging

Unsere Entscheidung: Jeder push auf `main` landet automatisch auf Staging.

Neue Arbeit startet immer vom neuesten `main` — also erst pull.

Was ist ein Branch?

Ein Branch ist eine **eigene Linie von Commits**. Ihr zweigt von `main` ab, arbeitet in Ruhe — und bringt eure Änderung per Pull Request zurück.



Warum?

- `main` bleibt stabil — geht nach jedem Merge auf Staging.
- Ihr stört niemanden, niemand stört euch.
- Ein Issue = ein Branch = ein PR: klein, gut zu reviewen.
- Schiefgegangen? Branch wegwerfen.

Gut zu wissen

- Kostet nichts: nur ein Name, der auf einen Commit zeigt.
- Name: Typ und Kurzbeschreibung, z. B. `feat/ada`.
- Entsteht lokal — GitHub sieht ihn erst nach dem push.
- Nach dem Merge wird er gelöscht.

🛠️ Etappe 2: Branch & Commit

Sagt eurem Agenten (ca. 10 Minuten)

1. **„Ich will eine Änderung machen. Leg dafür einen Branch nach AGENTS.md an.“**
2. **„Erstelle nun eine persönliche Karte — Name, Emoji, Motto.“**
3. **„Committe das. Noch nicht hochladen.“**
4. **„Zeige mir das Ergebnis lokal im Browser an“**

Selbst prüfen: `index.html` ist eure Karte da? Und auf GitHub: gibt es euren Branch dort schon?

Etappe 3: Push & Pull Request

Die Änderung geht zu GitHub — und ihr bittet um Aufnahme in `main`.

1 · Klonen | 2 · Branch & Commit | **3 · Push & PR** | 4 · Checks | 5 · Review | 6 · Merge & Staging | 7 · Release & Prod

Der PR-Titel wird zum Changelog

Der Titel des Pull Requests landet später **wörtlich** in den Release-Notes. Darum hat er eine feste Form:

```
typ: was sich ändert
```

Gut

```
feat: add anna to the
team wall
fix: keep long mottos
inside the card
```

Fällt durch die CI

```
Update team.js
fix stuff
```

Formal ok, sagt nichts

```
feat: changes
```

`feat` = neue Funktion · `fix` = Fehler behoben · `docs`, `chore`, `test` … = Sonstiges

🛠️ Etappe 3: Push & Pull Request

Sagt eurem Agenten (ca. 10 Minuten)

1. **„Lade meinen Branch hoch.“**
2. **„Erstelle einen Pull Request mit Conventional-Commit-Titel“**
3. **„Gib mir den Link zum PR.“**

Selbst prüfen: PR im Browser öffnen: Titel verständlich? Nur eure eine Zeile geändert?

Etappe 4: Checks

CI und KI-Review laufen automatisch bei jedem Pull Request.

1 · Klonen | 2 · Branch & Commit | 3 · Push & PR | **4 · Checks** | 5 · Review | 6 · Merge & Staging | 7 · Release & Prod

🛠️ Etappe 4: Checks

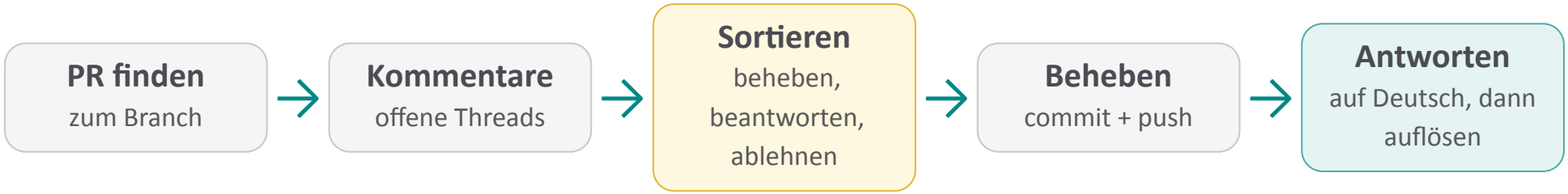
Sagt eurem Agenten (ca. 10 Minuten)

1. **„Sind die Checks in meinem PR grün?“**
2. **„Falls rot: Lies das Log und behebe den Fehler — ohne Regeln abzuschalten.“**
3. **„Fass das KI-Review zusammen. Was davon ist berechtigt?“**

Selbst prüfen: Jeden Kommentar beantworten und auflösen — vorher lässt sich nicht mergen. Rot ist gut: Der Fehler ist gefunden, bevor ihn jemand sieht.

Skill: `/pr-fix`

Ein **Skill** ist eine Anleitung als Markdown-Datei — der Agent arbeitet sie Schritt für Schritt ab. `/pr-fix` erledigt die offenen Review-Kommentare eures Pull Requests.



Eingebaute Bremsen

- Kein offener PR → tut nichts.
- Fragen werden beantwortet, aber nicht aufgelöst.
- Erst push, dann antworten — nie umgekehrt.
- Unklarer Kommentar → fragt euch statt zu raten.

Eure Rolle

- Die Sortierung prüfen, bevor er ändert.
- Ablehnen ist erlaubt — mit Begründung.
- Die Antworten stehen unter eurem Namen: lesen!

Aufruf auf eurem PR-Branch: `/pr-fix` — der Agent arbeitet, ihr prüft.

Etappe 5: Review

Vier Augen: Ein Mensch gibt frei — nie der Autor selbst.

1 · Klonen | 2 · Branch & Commit | 3 · Push & PR | 4 · Checks | **5 · Review** | 6 · Merge & Staging | 7 · Release & Prod

Gutes Review

Ihr als Reviewer:in

- Passt der PR zum Issue — und **nur** dazu?
- Selbst ansehen, nicht nur dem Agenten glauben
- Ist der Titel ein guter Changelog-Eintrag?
- Konkret kommentieren: **was** und **warum**

Der KI-Reviewer

- Prüft gegen `AGENTS.md`, schnell und gründlich
- Kennt aber euer Ziel nicht — und kann irren
- **Gibt nichts frei** — das macht immer ein Mensch

🛠️ Etappe 5: Review

Sagt eurem Agenten (ca. 10 Minuten)

1. **„Fordere für meinen PR ein Review von der Person an, die in meinem Issue steht.“**
2. **„Welche PRs soll ich reviewen?“**
3. **„Hol mir diesen PR auf den Rechner und fass zusammen: Passt er zum Issue und zu AGENTS.md?“**

Selbst prüfen: Den fremden PR im Browser ansehen und **selbst** freigeben oder Änderungen anfordern.

Etappe 6: Merge & Staging

Ab in `main` — und automatisch auf Staging.

1 · Klonen | 2 · Branch & Commit | 3 · Push & PR | 4 · Checks | 5 · Review | **6 · Merge & Staging** | 7 · Release & Prod

Merge-Konflikt — keine Panik

Zwei PRs ändern **dieselbe Stelle**. Wer zuerst merged, gewinnt — der andere muss nachziehen.

```
<<<<<<< main
  { login: "anna", … },
=======
  { login: "ben", … },
>>>>>>> mein-branch
```

Was richtig ist, wisst ihr

Git kann nicht entscheiden, welche Zeile gilt. Hier: **beide**, alphabetisch.

„Mein PR hat einen Konflikt mit main. Löse ihn so, dass beide Einträge erhalten bleiben, und erklär mir, was du gemacht hast.“

🛠️ Etappe 6: Merge & Staging

Sagt eurem Agenten (ca. 15 Minuten)

1. **„Ist mein PR freigegeben und grün? Dann merge ihn.“**
2. **„Hol den neuesten Stand von main auf meinen Rechner.“**
3. **„Lösch meinen alten Branch lokal.“**

Selbst prüfen: Nach ein paar Minuten `meko-christian.github.io/workshop-teamwand` öffnen: Seid ihr auf der Seite zu sehen?

Etappe 7: Release & Prod

Was auf Staging läuft, geht jetzt live.

1 · Klonen | 2 · Branch & Commit | 3 · Push & PR | 4 · Checks | 5 · Review | 6 · Merge & Staging | **7 · Release & Prod**

Release — Versionen aus PR-Titeln

Nach jedem Merge sammelt **release-please** alles in **einem** Release-PR:

- Changelog aus euren **PR-Titeln**
- neue Versionsnummer
- Merge des Release-PRs → **Prod**

Prod bekommt genau den Stand, der schon auf Staging lief.

chore(main): release 0.2.0

```
## 0.2.0

### Features
* add anna to the team wall (#9)
* add ben to the team wall (#11)
* add carla to the team wall (#12)
```

🛠️ Etappe 7: Release & Prod

Sagt eurem Agenten (ca. 15 Minuten)

1. **„Zeig mir den offenen Release-PR und fass den Changelog zusammen.“**
2. **„Ist die Release-Pipeline nach dem Merge durchgelaufen?“**

Selbst prüfen: Gemeinsam: Changelog verständlich? Freigeben und mergen — dann `meko-christian.github.io/workshop-teamwand` öffnen, Version unten auf der Seite prüfen.

Und wenn Prod kaputt ist?

Kein Hotfix per Hand: Änderung per Revert-PR zurücknehmen — gleicher Weg, auch unter Zeitdruck.

Das nehmt ihr mit

Der Ablauf

Klonen → Branch → Commit → Push → PR → Checks → Review → Merge → Staging → Release → Prod

Jede Änderung, jedes Mal. Gerade die kleinen.

Weiterüben

Issues mit Label `bonus`: kleine Features mit Test — gleicher Weg.

Für euer eigenes Projekt: Vorlage `MeKo-Tech/go-react-template`.

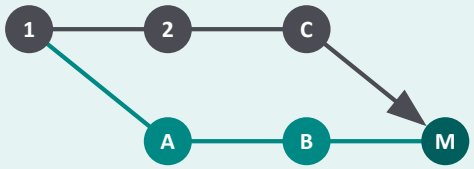
Der Agent tippt. Verantwortlich seid ihr — und das Review ist eure Unterschrift.

Anhang

Merge oder Rebase?

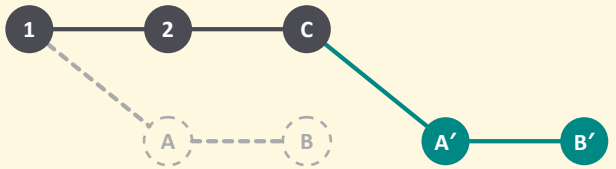
Euer Branch ist hinter `main` — inzwischen wurde `C` gemerged. Zwei Wege, ihn auf den neuesten Stand zu bringen:

Merge — zusammenführen



- `main` wird in euren Branch geholt, ein **Merge-Commit** `M` verbindet beide Linien.
- Eure Commits A, B bleiben, wie sie sind.
- History zeigt ehrlich, was wann passiert ist — mit Verzweigungen.

Rebase — neu aufsetzen



- Eure Commits werden **auf** das neue `main` neu abgespielt.
- Dabei entstehen **neue** Commits A′, B′ — die alten sind weg.
- History wird eine gerade Linie, als hättet ihr erst jetzt angefangen.

Der Code ist danach derselbe — nur die History sieht anders aus.

Merge oder Rebase — was heißt das für uns?

Auf GitHub: nur Squash-Merge

Der ganze PR wird **ein** Commit auf `main` — mit dem PR-Titel als Nachricht.

- `main` bleibt immer eine gerade Linie.
- Wie es in eurem Branch aussah, ist danach egal.

Im Branch: Merge ist der Standard

Euren Branch mit `main` abgleichen — beides geht:

- **Merge:** sicher, kein Umschreiben. Macht auch der Button „Update branch“.
- **Rebase:** schöner, aber danach braucht der push `--force-with-lease`.

Goldene Regel

Nie Commits rebasen, die andere schon haben — z. B. einen Branch, an dem zwei arbeiten. Wer seinen Stand schon geholt hat, bekommt sonst doppelte Commits. `main` ist deshalb gegen force push gesperrt.

„Bring meinen Branch auf den Stand von main — per Merge, nicht per Rebase.“